

CLOUD COMPUTING FUNDAMENTALS AND EVOLUTION

Module 1: From Servers to Services

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

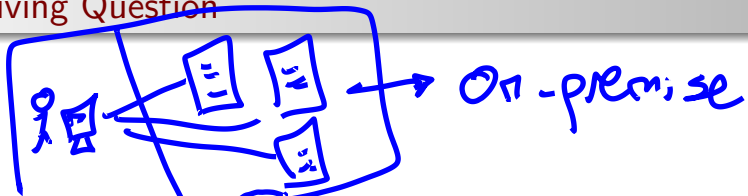
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-III



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Driving Question



**Why did we move from
buying servers to consuming
computing as a service?**



Outline

- 1 Session 1: What is Cloud Computing Really?
- 2 Session 2: From Mainframes to Cloud
- 3 Session 3: Why Cloud Computing?
- 4 Session 4: Conceptual Cloud Architecture



Outline

scrum → sprint — cycle
 waterfall

1 Session 1: What is Cloud Computing Really?

M } Planning (Backlog)
 8am

T } Dailies / Pull Branches
 W

T/mon. } Develop Branch → Test Branch (QA)
 4 Session 4: Conceptual Cloud Architecture

F } Review

Client
 Retrospective

new task current sprint

feedback

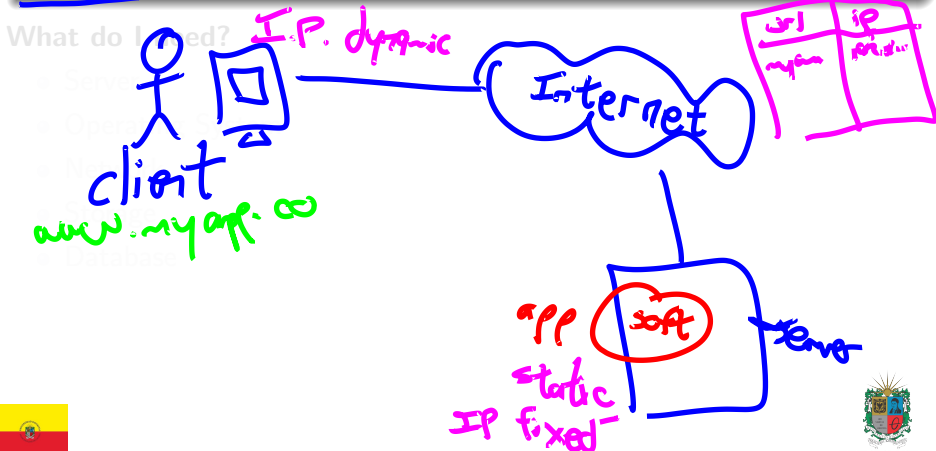


A Seemingly Simple Question

Context

I have a web application
I need to run it.

What do I need?



A Seemingly Simple Question

Context

I have a web application.

I need to run it.

What do I need?

- Server
- Operating System
- Network
- Storage
- Database

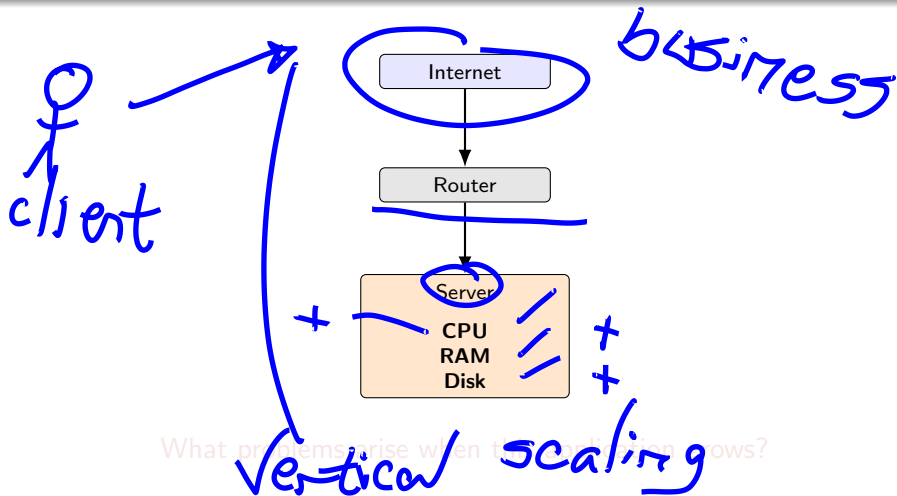
buy
cost ← maintenance
format + setup
no GUI (networks)
(security)
Firewall + SSL +
double ISP

SSD → OS

HDD → Data



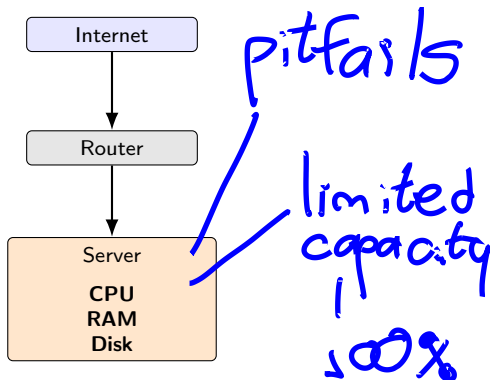
The Traditional Server



What problems arise when the application grows?



The Traditional Server



What problems arise when this application grows?

queue

← available



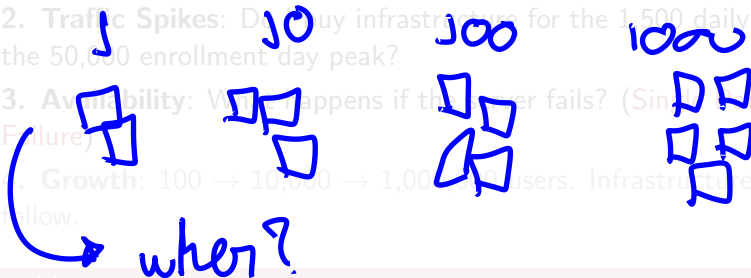
The Four Problems

- 1 **Capacity Planning**: How many servers do I buy as users increase over time?

- 2. **Traffic Spikes**: Do I buy infrastructure for the 1,500 daily users or the 50,000 enrollment day peak?

- 3. **Availability**: What happens if the server fails? (Single Point of Failure)

- 4. **Growth**: 100 → 10,000 → 1,000,000 users. Infrastructure must follow.



A New Idea

What if infrastructure could be a service?

Instead of buy → install → configure → maintain

We can request → use → scale → pay for consumption



The Four Problems

- 1. **Capacity Planning:** How many servers do I buy as users increase over time?
- 2. **Traffic Spikes** Do I buy infrastructure for the 1,500 daily users or the 50,000 enrollment day peak?
- 3. **Availability:** What happens if the server fails? (Single Point of Failure)
- 4. **Growth:** 100 → 10,000 → 1,000,000 users. Infrastructure must follow

~~Amazon~~
Amazon → AWS



The Four Problems

- **1. Capacity Planning:** How many servers do I buy as users increase over time?
 - **2. Traffic Spikes:** Do I buy infrastructure for the 1,500 daily users or the 50,000 enrollment day peak?
 - **3. Availability:** What happens if the server fails? (Single Point of Failure)
 - **4. Growth:** 100 → 10,000 → 1,000,000 users. Infrastructure must follow
- ↳ risk!

recovery plan?

A New Idea

What if infrastructure could be a service?

Instead of buy → install → configure → maintain

We can request → use → scale → pay for consumption



The Four Problems

- **1. Capacity Planning:** How many servers do I buy as users increase over time?
- **2. Traffic Spikes:** Do I buy infrastructure for the 1,500 daily users or the 50,000 enrollment day peak?
- **3. Availability:** What happens if the server fails? (Single Point of Failure)
- **4. Growth:** 100 → 10,000 → 1,000,000 users. Infrastructure must follow.

scalability

VPN

A New Idea

What if infrastructure could be a service?

Instead of buy → install → configure → maintain

We can request → use → scale → pay for consumption

client

cloud
DaaS

internal static ip

ip dynamical



The Four Problems

- **1. Capacity Planning:** How many servers do I buy as users increase over time?
- **2. Traffic Spikes:** Do I buy infrastructure for the 1,500 daily users or the 50,000 enrollment day peak?
- **3. Availability:** What happens if the server fails? (Single Point of Failure)
- **4. Growth:** 100 → 10,000 → 1,000,000 users. Infrastructure must follow.

1st public cloud → AWS

A New Idea

What if infrastructure could be a service?

IaaS

Instead of buy → install → configure → maintain

We can request → use → scale → pay for consumption

\$ / hour → 10-10 USD → 1000 hrs



NIST Definition of Cloud Computing

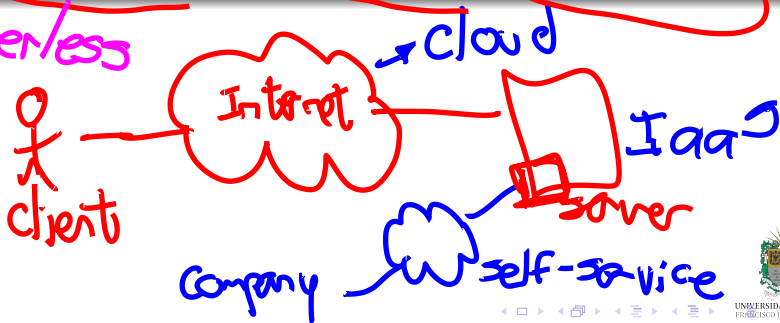
Five Essential Characteristics

- 1 **On-demand self-service**: Provision resources directly.
- 2 **Broad network access**: Accessible via standard networks.
- 3 **Resource pooling**: Physical resources shared among multiple clients.
- 4 **Rapid elasticity**: Resources can grow/shrink dynamically.
- 5 **Measured service**: Consumption is metered (pay-as-you-go).

minutes

scalability

serverless



Scalability vs. Elasticity

Scalability

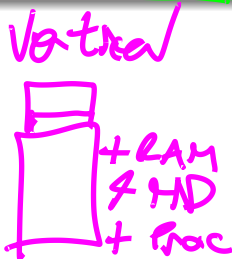
Capacity to support growth by adding resources.

Elasticity

Capacity to **adapt dynamically** to demand (up and down).

Cloud $\neq$ Internet

Internet is a **network**. Cloud Computing is a **model** for providing computational resources as services.

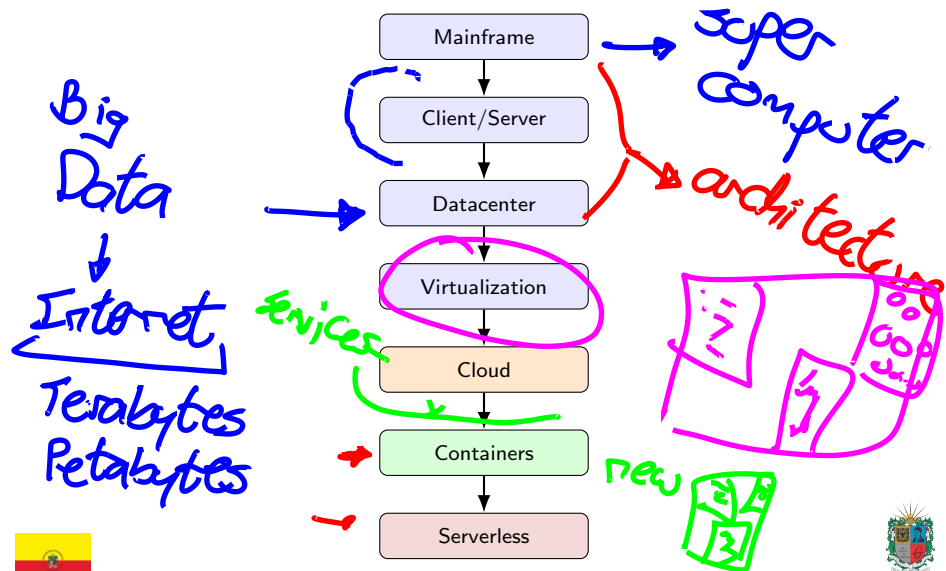


Outline

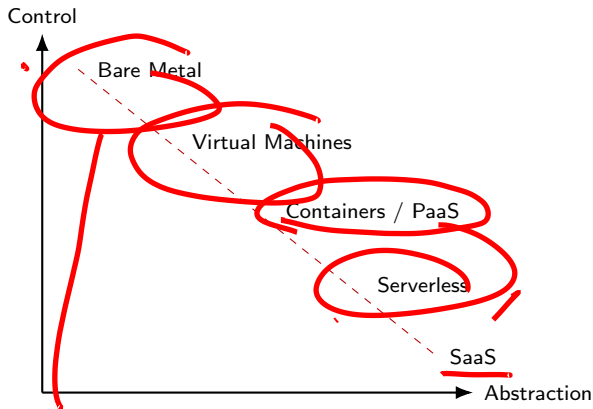
- 1 Session 1: What is Cloud Computing Really?
- 2 Session 2: From Mainframes to Cloud
- 3 Session 3: Why Cloud Computing?
- 4 Session 4: Conceptual Cloud Architecture



Technological Evolution



The Abstraction Staircase



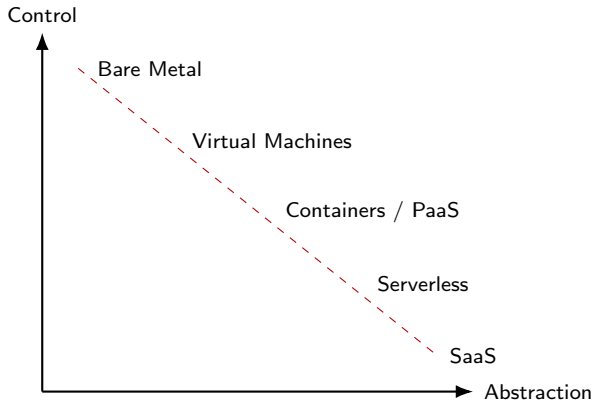
Trade-off

As we go up: less administration, faster time-to-market.

As we go down: more control, custom configurations.

management

The Abstraction Staircase



Trade-off

As we go up: less administration, faster time-to-market.

As we go down: more control, custom configurations.

Outline

clinic
story

1 Session 1: What is Cloud Computing Really?

2 Session 2: From Mainframes to Cloud

3 Session 3: Why Cloud Computing?

4 Session 4: Conceptual Cloud Architecture

Kali

Linux

on-premise
sq

sniffer



CAPEX vs OPEX

on-prem. s.

CAPEX (Capital Expenditure)

Buying infrastructure ↓

Large initial investment ↓

Owned hardware

OPEX (Operational Expenditure)

Consuming as a service ↓

Operational expense ↓

Pay exactly for usage

The Sizing Problem

Overprovisioning: Buying for the worst case → Wasted resources.

Underprovisioning: Buying too little → Lost users, bad experience.



CAPEX vs OPEX

CAPEX (Capital Expenditure)

Buying infrastructure ↓
 Large initial investment ↓
 Owned hardware

OPEX (Operational Expenditure)

Consuming as a service ↓
 Operational expense ↓
 Pay exactly for usage

The Sizing Problem

Overprovisioning: Buying for the worst case → Wasted resources.

Underprovisioning: Buying too little → Lost users, bad experience.

scaling



The Business Case

Advantages:

- Economies of Scale
- Fast Time to Market (minutes vs months)
- Access to Innovation (AI, ML, specialized DBs)
- Global Reach

But Cloud isn't magic:

- Complexity
- Vendor Lock-in
- Hidden Costs (network egress, backups, requests)



The Business Case

Advantages:

- Economies of Scale
- Fast Time to Market (minutes vs months)
- Access to Innovation (AI, ML, specialized DBs)
- Global Reach

But Cloud isn't magic:

- Complexity
- Vendor Lock-in
- Hidden Costs (network egress, backups, requests)



Outline

1 ~~Session 1: What is Cloud Computing Really?~~

2 ~~Session 2: From Mainframes to Cloud~~

3 ~~Session 3: Why Cloud Computing?~~

4 Session 4: Conceptual Cloud Architecture



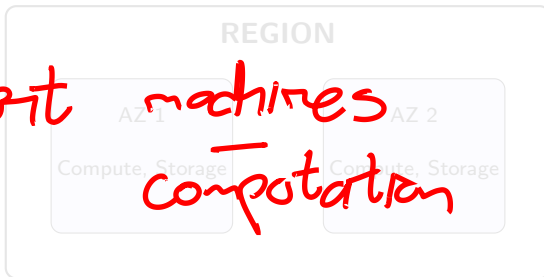
Where is "the cloud"?

→ datacenter ^{LB} ^{San Polo}

Definition

Cloud = Physical Infrastructure + Software + Networks + Automation + Services

- **Region.** Geographic area containing cloud infrastructure.
- **Availability Zone (AZ):** Isolated data centers within a region.

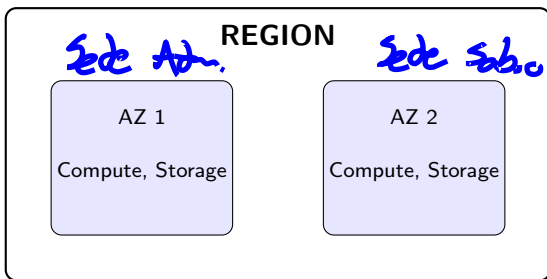


Where is "the cloud"?

Definition

Cloud = Physical Infrastructure + Software + Networks + Automation + Services

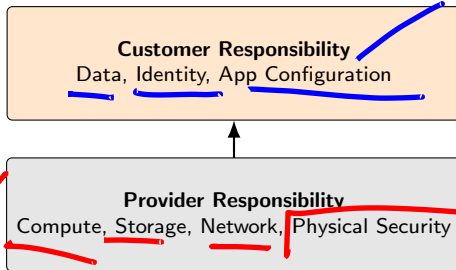
- **Region:** Geographic area containing cloud infrastructure.
- **Availability Zone (AZ):** Isolated data centers within a region.



North
Virginia



Shared Responsibility Model



update

multi-region

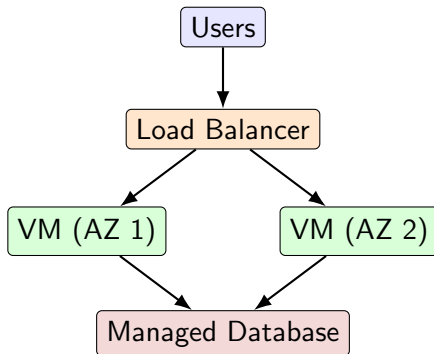
Control Plane vs Data Plane

Control Plane: API/Console to manage resources.

Data Plane: Execution of user traffic, VMs, and data processing.



Architectural Evolution



Redundancy + Distribution = **High Availability**



Outline

